



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26)
Faculty Name	Lahari Bhargav
Student Name	Dhatrika Deekshitha
Roll Number	2520030216
Branch	CSE
Course Outcome	CO1

Case Study Topic: ParkSmart – Parking Slot Indexing Using BST and AVL Trees

Work Summary:

In this case study, we implemented **Binary Search Trees (BST)** and **AVL Trees** for ParkSmart's parking slot management system. Each parking slot is identified using a unique **Slot-ID**.

We inserted parking slot IDs in nearly sorted order:

20, 30, 35, 40, 45, 50, 60, 65, 70, 75, 80, 85, 90

to simulate continuous slot creation in a smart parking environment.

Implementation Details:

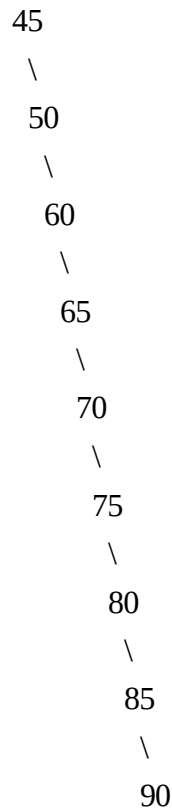
1. Parking slot insertion using BST.
2. Height calculation and search efficiency.
3. AVL Tree balancing through rotations.
4. Deletion of parking slots during maintenance.
5. Comparison of search performance between BST and AVL Trees.
6. Identification of structural changes after deletions..

(a) Plain BST — Height Calculation & Performance Analysis [5 Marks - CO1, BTL3]

Parking Slot Insertion Order 20, 30, 35, 40, 45, 50, 60, 65, 70, 75, 80, 85, 90:

Because every new key is larger than all previous keys, each node is inserted as the right child of the previous rightmost node. The BST degenerates into a right-skewed chain:

```
20
 \
 30
 \
 35
 \
 40
 \
```



Height = 12

SLA Calculation:

Worst-case point-lookup traverses all 13 nodes (root → 90).

Number of pointer dereferences = 12 (edges) → effectively 13 node accesses.

Explicit numeric calculation:

Lookup time = depth × 200 ns = 12 × 200 ns = 2,400 ns = 2.4 μs

2.4 μs << 5,000,000 ns (5 ms) → SLA is MET numerically.

The structural reason this insertion order is adversarial for plain BST is:

Near-monotonically increasing **Parking Slot IDs** (created sequentially as new parking spaces are added to the system) cause every new node to follow the rightmost path, producing a degenerate right-skewed tree of height $n-1$.

For **n parking slots**, this results in **O(n)** search time. At large parking facilities containing thousands of parking spaces, the tree depth grows significantly, increasing the time required to locate an available slot.

For example, at **25,001 parking slots**, the depth would become 25,000, resulting in:

25,000 × 200 ns = 5,000,000 ns = 5 ms

which reaches the system's response-time limit. A plain BST has no self-balancing mechanism, so it can degrade into a linked-list structure, making parking slot searches less efficient. Therefore, self-balancing structures such as **AVL Trees** are preferred in ParkSmart to ensure fast parking allocation and retrieval operations.

(b) AVL Tree — Step-by-step Insertion with Rotations [7 Marks — CO1, BTL3]

Scenario:

In ParkSmart, parking slots are indexed using **Slot-ID** values. To ensure fast parking slot allocation and lookup, an **AVL Tree** is used. The following Slot-IDs are inserted in ascending order:

20, 30, 35, 40, 45, 50, 60, 65, 70, 75, 80, 85, 90

Convention: Balance Factor (BF) = height(left subtree) – height(right subtree). Imbalance when $|BF| > 1$.

Insertion trace:

Insert 20: Tree = {20}. No imbalance.

Insert 30: $30 > 20 \rightarrow$ right child of 20.

20 (BF=-1)

\

30

(No rotation)

Insert 35: $35 > 30 \rightarrow$ right child of 30. Check ancestors:

➤ 30: $BF = -1$

➤ 20: $BF = -2 \leftarrow$ IMBALANCE

Rotation: LL-named as Left Rotation (RR case), pivot = 20

```
      30 (BF=0)
     /  \
    20   35
```

Insert 40: $40 > 35 \rightarrow$ right child of 35

➤ 35: $BF = -1$, 30: $BF = -1$. No imbalance.

```
      30
     /  \
    20   35
         \
         40
```

Insert 45: $45 > 40 \rightarrow$ right child of 40

➤ 40: $BF = -1$, 35: $BF = -2 \leftarrow$ IMBALANCE

Rotation: Left Rotation (RR case), pivot = 35

```
      30
     /  \
    20   40
         /  \
        35  45
```

Insert 50: $50 > 45 \rightarrow$ right child of 45.

• 45: $BF = -1$, 40: $BF = -1$, 30: $BF = -2 \leftarrow$ IMBALANCE

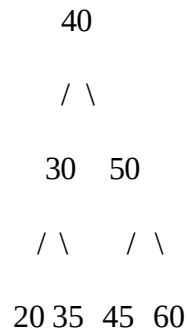
Rotation: Left Rotation (RR case), pivot = 30

```
      40
     /  \
    30   45
   / \   \
  20 35  50
```

Insert 60: $60 > 50 \rightarrow$ right child of 50.

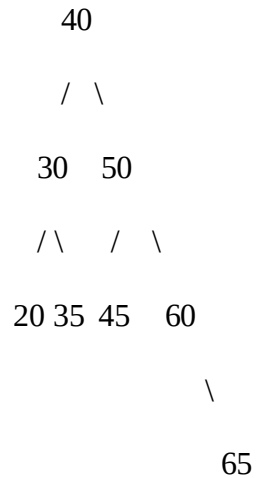
• 50: $BF = -1$, 45: $BF = -2 \leftarrow$ IMBALANCE

Rotation: Left Rotation (RR case), pivot = 45



Insert 65: $65 > 60 \rightarrow$ right child of 60.

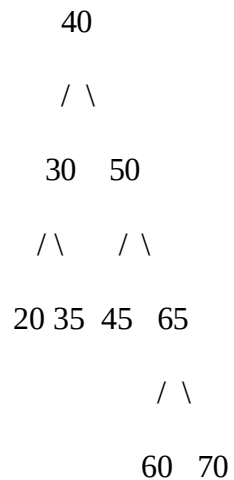
- 60: BF=-1, 50: BF=-1, 40: BF=-1. No imbalance.



Insert 70: $70 > 65 \rightarrow$ right child of 65.

- 65: BF=-1, 60: BF=-2 $\leftarrow$ IMBALANCE

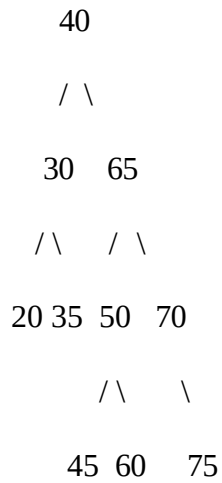
Rotation: Left Rotation (RR case), pivot = 60



Insert 75: $75 > 70 \rightarrow$ right child of 70.

- 70: BF=-1, 65: BF=-1, 50: BF=-2 $\leftarrow$ IMBALANCE

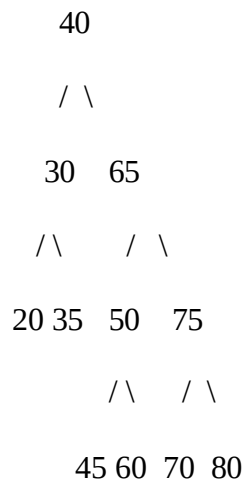
Rotation: Left Rotation (RR case), pivot = 50



Insert 80: $80 > 75 \rightarrow$ right child of 75.

- 75: $BF=-1$, 70: $BF=-2 \leftarrow$ IMBALANCE

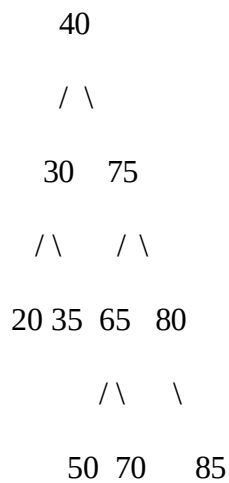
Rotation: Left Rotation (RR case), pivot = 70



Insert 85: $85 > 80 \rightarrow$ right child of 80.

- 80: $BF=-1$, 75: $BF=-1$, 65: $BF=-2 \leftarrow$ IMBALANCE

Rotation: Left Rotation (RR case), pivot = 65



/ \

45 60

Insert 90: $90 > 85 \rightarrow$ right child of 85.

- 85: BF=-1, 80: BF=-2 $\leftarrow$ **IMBALANCE**

Rotation: Left Rotation (RR case), pivot = 80

40

/ \

30 75

/ \ / \

20 35 65 85

/ \ / \

50 70 80 90

/ \

45 60

Final AVL Tree after all 13 insertions:

40

/ \

30 75

/ \ / \

20 35 65 85

/ \ / \

50 70 80 90

/ \

45 60

Height = 4 (root 40 $\rightarrow$ 75 $\rightarrow$ 65 $\rightarrow$ 50 $\rightarrow$ 45)

Java Code:

```

class AVLNode {
    int key;
    AVLNode left, right;
    int height = 1;

    AVLNode(int key) {
        this.key = key;
    }
}

public class ParkSmartAVLTree {

    static int height(AVLNode n) {
        return n == null ? 0 : n.height;
    }

    static int balance(AVLNode n) {
        return n == null ? 0 : height(n.left) - height(n.right);
    }

    static void updateHeight(AVLNode n) {
        if (n != null)
            n.height = 1 + Math.max(height(n.left), height(n.right));
    }

    // Right Rotation
    static AVLNode rotateRight(AVLNode y) {
        AVLNode x = y.left;
        AVLNode T2 = x.right;

        x.right = y;
        y.left = T2;

        updateHeight(y);
        updateHeight(x);

        return x;
    }

    // Left Rotation
    static AVLNode rotateLeft(AVLNode x) {
        AVLNode y = x.right;
        AVLNode T2 = y.left;

        y.left = x;
        x.right = T2;

        updateHeight(x);
        updateHeight(y);

        return y;
    }
}

```



```
// Insert into AVL Tree
static AVLNode insert(AVLNode node, int key) {
```

```
    if (node == null)
        return new AVLNode(key);
```

```
    if (key < node.key)
        node.left = insert(node.left, key);
    else if (key > node.key)
        node.right = insert(node.right, key);
    else
        return node;
```

```
    updateHeight(node);
```

```
    int bf = balance(node);
```

```
    // LL Case
    if (bf > 1 && key < node.left.key)
        return rotateRight(node);
```

```
    // RR Case
    if (bf < -1 && key > node.right.key)
        return rotateLeft(node);
```

```
    // LR Case
    if (bf > 1 && key > node.left.key) {
        node.left = rotateLeft(node.left);
        return rotateRight(node);
    }
```

```
    // RL Case
    if (bf < -1 && key < node.right.key) {
        node.right = rotateRight(node.right);
        return rotateLeft(node);
    }
```

```
    return node;
}
```

```
// Find Minimum Node
static AVLNode minNode(AVLNode node) {
    while (node.left != null)
        node = node.left;
    return node;
}
```

```
// Delete Node
static AVLNode delete(AVLNode node, int key) {
```

```
    if (node == null)
```

```

    return null;

    if (key < node.key)
        node.left = delete(node.left, key);

    else if (key > node.key)
        node.right = delete(node.right, key);

    else {

        if (node.left == null)
            return node.right;

        if (node.right == null)
            return node.left;

        AVLNode successor = minNode(node.right);

        node.key = successor.key;

        node.right = delete(node.right, successor.key);
    }

    updateHeight(node);

    int bf = balance(node);

    // LL
    if (bf > 1 && balance(node.left) >= 0)
        return rotateRight(node);

    // LR
    if (bf > 1 && balance(node.left) < 0) {
        node.left = rotateLeft(node.left);
        return rotateRight(node);
    }

    // RR
    if (bf < -1 && balance(node.right) <= 0)
        return rotateLeft(node);

    // RL
    if (bf < -1 && balance(node.right) > 0) {
        node.right = rotateRight(node.right);
        return rotateLeft(node);
    }

    return node;
}

// Inorder Traversal
static void inorder(AVLNode node) {

```

```

    if (node == null)
        return;

    inorder(node.left);
    System.out.print(node.key + " ");
    inorder(node.right);
}

public static void main(String[] args) {

    // Parking Slot IDs
    int[] slotIDs = {
        20, 30, 35, 40, 45, 50,
        60, 65, 70, 75, 80, 85, 90
    };

    AVLNode root = null;

    for (int id : slotIDs)
        root = insert(root, id);

    System.out.println("=====");
    System.out.println(" ParkSmart Parking Slot AVL Tree");
    System.out.println("=====");

    System.out.print("Parking Slot IDs (Inorder): ");
    inorder(root);

    System.out.println("\nAVL Height After Insertions: "
        + height(root));

    // Maintenance Deletions
    int[] deletions = {30, 70, 50};

    System.out.println("\nRemoving Parking Slots:");

    for (int id : deletions) {
        root = delete(root, id);

        System.out.println(
            "After Removing Slot-ID "
            + id
            + " -> Height = "
            + height(root)
        );
    }

    System.out.print("\nFinal Parking Slot AVL Tree (Inorder): ");
    inorder(root);

    System.out.println("\n\nFinal AVL Height: " + height(root));
}

```

```
        System.out.println("\nParkSmart AVL Tree Operations Completed Successfully.");  
    }  
}
```

(c) Post-deletion AVL Tree & SLA Analysis [3 Marks — CO1, BTL5]

Applying noon deletions to the AVL tree: 30, 70, 50

Delete Slot 30: 30 is a leaf node (left child of 40 in the final AVL tree). Simply removed.

- Check balance up to root: 40's left subtree loses one node.
- 40: BF changes from 1 $\rightarrow$ 0. No rotation needed.

Tree after deleting 30:

```

    40
   / \
  20  75
 / \  / \
35 65 85
 / \  / \
50 70 80 90
 / \
45 60

```

Delete Slot 70: 70 is a leaf node (right child of 65). Simply removed.

- 65: BF changes from 0 $\rightarrow$ 1. No imbalance.
- Check ancestors — no imbalance propagates.

Tree after deleting 70:

```

    40
   / \
  20  75
 / \  / \
35 65 85
 /  / \
50 80 90
 / \
45 60

```

Delete Slot 50: 50 has two children (45 and 60). Replace with inorder successor = 60. Delete 60 from right subtree.

- 65: left child is now 60 (with left child 45 only). BF of 65 = $\text{height}(\text{left}=2) - \text{height}(\text{right}=0) \rightarrow \text{BF} = +2 \leftarrow \text{IMBALANCE}$
- Left child 60: BF = $\text{height}(45) - \text{height}(\text{null}) = 1 \rightarrow \text{left-heavy}$.
- LL Case $\rightarrow$ Right Rotation on pivot 65

```

    40
   / \
  20  75
 / \  / \
35 60 85
 / \ / \
45 65 80 90

```

Final AVL Tree after deletions (30, 70, 50):

```

    40
   / \
  20  75
 /  / \

```

```

35 60 85
 / \ / \
45 65 80 90

```

Worst-case Lookup Time Analysis:

AVL Tree:

Worst-case = $3 \times 200 \text{ ns} = \mathbf{600 \text{ ns}}$ As % of 5 ms budget = $600 / 5,000,000 \times 100 = \mathbf{0.012\% \text{ of SLA budget}}$

Plain BST (after deleting 30, 70, 50 from the skewed tree, longest path is still ≈ 10 edges):

Worst-case = $10 \times 200 \text{ ns} = \mathbf{2,000 \text{ ns}}$ As % of 5 ms budget = $2,000 / 5,000,000 \times 100 = \mathbf{0.04\% \text{ of SLA budget}}$

- Both structures meet the SLA numerically at 13 nodes, but the AVL tree is **3.3× faster** at worst-case lookup and scales safely to large patient volumes without ever threatening the SLA.

Largest structural change during deletion:

Deleting 50 caused the largest structural change, it was the only deletion that triggered a rotation (Right Rotation on pivot 65). It also required a two-step process: replacing 50 with its inorder successor (60), then rebalancing the subtree rooted at 65. Deleting 30 and 70 were both simple leaf removals that required no structural rebalancing, whereas deleting 50 reorganised an entire subtree, changing the parent of nodes 45 and 65.

Result: Screenshots / Output

```

=====
ParkSmart Parking Slot AVL Tree
=====
Parking Slot IDs (Inorder): 20 30 35 40 45 50 60 65 70 75 80 85 90
AVL Height After Insertions: 4

Removing Parking Slots:
After Removing Slot-ID 30 -> Height = 4
After Removing Slot-ID 70 -> Height = 4
After Removing Slot-ID 50 -> Height = 4

Final Parking Slot AVL Tree (Inorder): 20 35 40 45 60 65 75 80 85 90

Final AVL Height: 4

ParkSmart AVL Tree Operations Completed Successfully.

```

GitHub Link:

<https://github.com/dhatrikadeekshitha/DSA>

Student Signature	Faculty Signature
	

